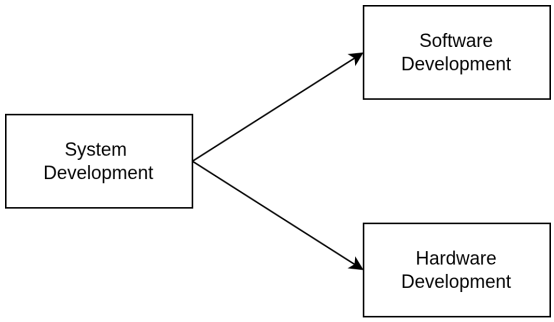
1 Introduction

1.1 History

1.1.1 System Development

System Development

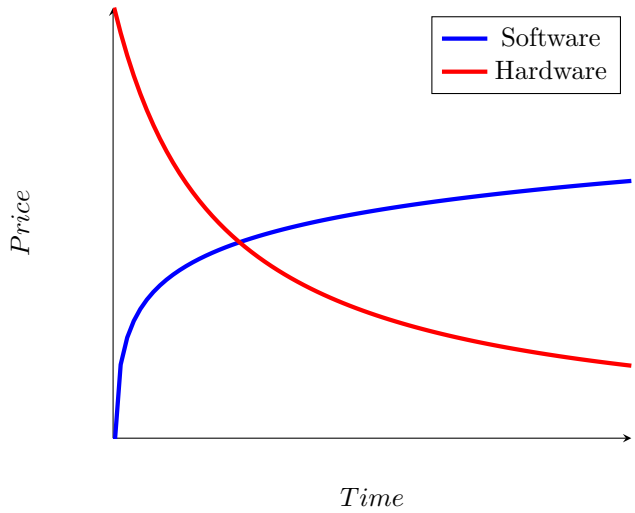
In the 1950s, we did not talk about software development but rather system development, which involved both hardware and software. It was difficult to develop both hardware and software at the same time, so these phases were divided and decoupled.



1.1.2 Software Crisis

Software Crisis

Even though hardware became more abundant and available, creating software was still difficult because projects could not be delivered on time, often went over budget, and were filled with bugs. This is what we call the software crisis, and it was mainly due to a lack of methodology in software development.

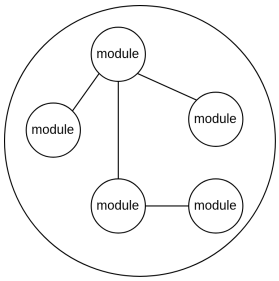


As shown in the graph above, there is an inverse relationship between hardware and software prices. As hardware becomes more affordable and advanced, software prices tend to increase.

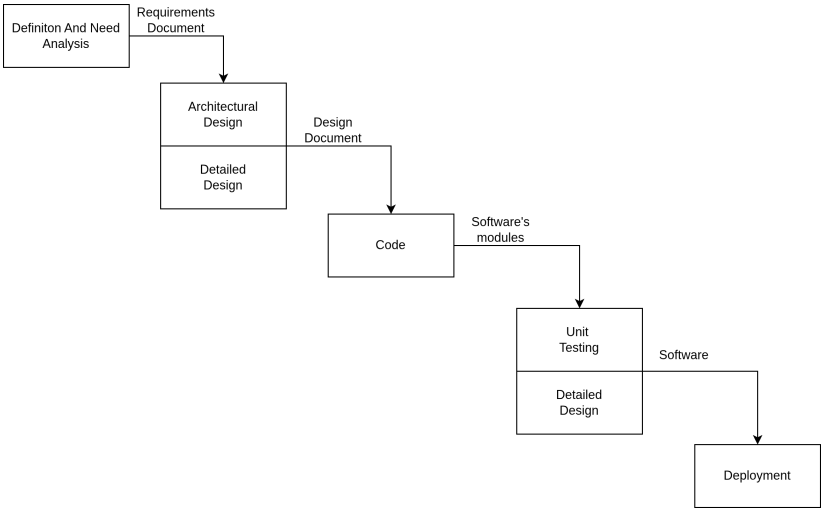
1.1.3 Life Cycle

Life Cycle

The way we develop software is very important. To address the software crisis, engineers developed life cycles, which are methodologies that help build software through chronological steps. One of the first life cycles still being used is the waterfall life cycle.
Furthermore, system problems are divided into smaller, more manageable subproblems called modules, then linked together (architectural design), and each module is then detailed (detailed design).



Architectural Design



WaterFall Life Cycle

1.1.4 Process Management

Process Management

In the 1980s, there was a growing interest in process management across many fields, e.g., how to organize medical processes in hospitals. With this came the ISO 9000 standard in 1987 to frame quality management regardless of the field.
Therefore, software engineers started to create standards specific to software quality.

1.1.5 Why Life Cycles Aren't Enough?

Why Life Cycles Aren't Enough?

Even though life cycles provide a methodology, they are a bit too abstract and not detailed enough, e.g., how to go from architectural design to detailed design? How should tasks be organized? etc.

1.2 Type Of Software Process

Type

The complexity of the process is tied to the complexity of the project, so we need to choose according to the context of the project. With hardware becoming more abundant, software became smaller, and from this came lighter processes such as Agile. In total, there are three types of processes:

- **Light-weight**: Agile
- **Heavy-weight**: classical processes
- **Hybrid**: use heavy-weight processes for complex phases and light-weight ones for simpler phases

Note

- Any organization uses processes wether implicit or explicit but we are only interested in explicits one.
- Agile isn't magical solution for every project it has its limitation like any other process (not suited for critical project, works only for small teams, ...etc)

1.3 Examples Of Software Processes

Examples

- **Agile in the Large**: Agile methods applied to large, complex projects.
- **Global & Distributed Software Processes**: divide a project into subprojects, each managed independently by different teams, often distributed geographically. The main challenges are communication and integration between subprojects.
- **Following the Sun (Around the Clock)**: similar to distributed development, but teams are organized across time zones so that work continues continuously, 24/7.

1.4 Configuration Software Rather Than Development

Configuration Software Rather Than Development

Let freedom to user to configure the software, to turn off/on features and to change parameters to abide to a wider clientele like ERP.

1.5 Software Processes & Software Development

Definition

Software development is a complexe task consisting of many inidvidual activities that need to be coordinated and this is where software processes come into place providing an infrastructure to coordinate & manage these various activities

Note

If development organization doesn't explicitly define its process it still uses them at least implicitly.

1.6 Why Use Software Process?

Why Use Software Process?

- simplification of understanding & communication between organizations
- support process improvement
- support processes management
- automate execution of some tasks
- automate process guidance
- using defined process makes us more reliable and trustworthy for the client
- a means to convince third parties of the quality of the software (good quality software process leads to good quality software)
- process for risk management
- process in small development effort
- standardization of processes: follow the norms & standards for guaranteed process quality
- industry & technologies trends :
  - **pros**: trends always bring something new
  - **cons**: we need to adapt the process accordingly
- big organizations use them explicitly but smaller ones we use really simple process to not encumber and make development harder
- We have to adapt the processes to the risk management.

Conclusion

Software processs variety changes according to context, complexity, standards followed, risk management, trends which complicate the management of the processes so this is why we have a field called **software processes**

1.7 Defined & Performed Process

1.7.1 Defined Process

Defined Process

It's **theoritical ideal process** defining all activities, technologies while following the standards.

1.7.2 Performed Process

Performed Process

What we actually **apply in real life** we have constraints, sometimes people don't 100% follow the process, they may have a missunderstanding, or they don't do their work properly

Note

The performed process can never be 100% identical to the defined process, we try to minimize the cost by continuous control.

1.8 Process Selection & Tailoring

Process Selection & Tailoring

- **Process Selection**: select the appropriate process according to the context.
- **Process Tailoring**: adapt the process so it fits 100%.

1.9 Process & Product Quality

Process & Product Quality

A **good process leads to a good product/context** , but sometimes a good process used in project A produced a good product but if used in another project B it **might not give as good of a result as A**.

1.10 Software Process Improvement (Continious)

1.10.1 Software process & Knowledge Management

Software process & Knowledge Management

Knowledge gained from projects can be used to improve the software process.

1.10.2 Software Process & People

Software Process & People

The process depends on the human factor, and humans are not predictable (motivation, creativity, etc.), which influences the process.

1.10.3 Process (Broadly Speaking)

Process ISO 9000 Definition

a set of interlaced or interacting activities which use inputs to deliver an intended result.

1.10.4 Task

Task ISO 12207 Definition

Required , recommanded or permissible action intended to contribute to the achievement of one or more outcomes of a process

1.10.5 Activity

Activity ISO 12207 Definition

a set of cohesive tasks of a process

1.10.6 Sub-Process

Sub-Process CMMI Definition

a process that is part of a larger process

1.10.7 Process Model

Process Model

Description of a model.

1.10.8 Process Instance

Process Instance ISO 33001 Definition

Single specific & identifiable execution of a process

1.10.9 Role

Role

A set of qualification and responsibilities played by actors (human , software , AI ...etc), on actor may play multiple roles at the same time.

1.10.10 Product (Artifact)

Product PMBOK Definition

That is produced is quantifiable and can be either an end item in itself or a componant item

1.11 Model (Broadly Speaking)

Model

Abstract representation of an existing entity or an entity to be created.

1.12 Type Of Model

1.12.1 Descriptive & Prescriptive Model

Descriptive & Prespective Model

- **Descriptive Model** : if a model represents an **existing entity** , e.g : biology describes existing creatures.
- **Prescriptive Model** : if it represents an **entity to be created** , e.g : blueprint of building prescribes an entity to be created.

1.12.2 Difference Between Descriptive & Prescriptive Model

Difference

The difference resides solely in the **purpose but not necessarily in the content** of the model.

1.13 Software Process Model

Software Process Model

- **Describes The Way Software Is To Be Developed** : in this case the software doesn't exist already therefore it's a **prescriptive model** to design the entity to be created.
- **Describes How The Software Is Actually Developed** : the software already exists so it's a **descriptive model**, its purpose is to **analyse & improve this process**.

Note

Software processes have the property of utilizing both descriptive & prescriptive models.

1.14 Stachowiak's General Model Theory (GMT)

GMT

In this theory the model is characterized by 3 properties :

- **Mapping Property** : a model is **mapping or representation** of some entity.
- **Reduction Property** : a model is a **reduction of the represented entity** and **doesn't contain all attributes** of the original (**an abstraction**).
- **Pragmatic Property** : a model is created for a **certain purpose** which implies that **the perfect model doesn't exist**, moreover one entity **may have different models** each contributing to a different purpose e.g building blueprints : architectural (layout/design), structural (foundation/framing), MEP (mechanical, electrical, plumbing).

1.15 Meta Model

Meta Model

In order to describe a model we need to define **how is to be done** and what terminologies may be used, in other words we need a model for the model (a meta model).

1.16 ISO Meta Model Definition

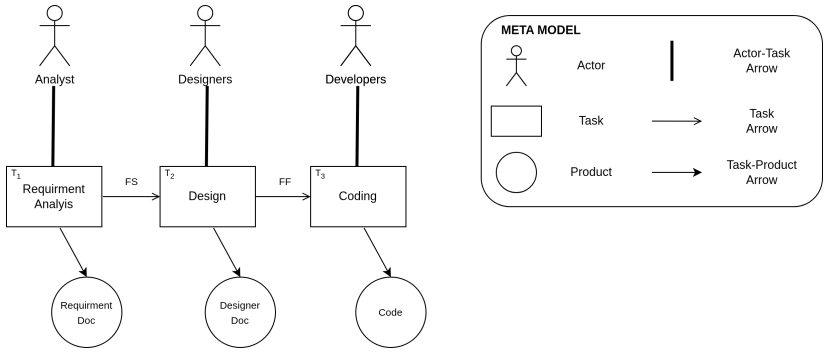
1.16.1 ISO 24744

ISO 24744

Meta model is specification of the concept relationships on the rules that are used to define a mythodology.

1.16.2 ISO 15909

ISO 15909

Meta model is a model defining the concept and the relation for some modeling notation.

1.17 Simple Example Of Meta Model